CampusConnect

A Smart Campus Exchange Platform

System Design Document

Architecture . Database Schema . Component & Class Diagrams . REST API Contracts (OpenAPI) . UI/UX Mockups

Team Members

Name (Enrollment No.)	Role
Dev Barot (202512069)	Project Manager & System Architect
Dhrumil Doshi (202512088)	Authentication & User Management Lead
Nishka Shah (202512035)	Marketplace & Listing Management Lead
Heer Mehta (202512095)	AI & Intelligent Services Lead
Heer Shah (202512024)	Search, Chat & Notification Lead
Charmi Bhayani (202512028)	Rental & Transaction Management Lead
Kajal Varlani (202512017)	Admin Panel & Analytics Lead
Deep Boghara (202512076)	Database, DevOps & Quality Assurance Lead

Course Instructor: JayPrakash Lalchandani

Document Type: System Design Document (SDD)

1. Introduction

1.1 Purpose

This System Design Document (SDD) translates the functional and non-functional requirements defined in the CampusConnect Requirements Specification (BRD) into a concrete technical design. It describes the system architecture, database schema, component and class structure, REST API contracts, and the final UI/UX mockups that the development team will implement across the 14-week execution plan defined in the Project Plan.

1.2 Scope

This document covers the design of the student-facing web client, the administrator dashboard, and the supporting backend, AI, and real-time services, consistent with the scope defined in the Project Proposal: a web-based marketplace exclusive to verified university students supporting buy, sell, rent, exchange, and giveaway workflows, enhanced with AI-assisted listing creation, semantic matching, OCR duplicate detection, real-time chat/notifications, QR-verified rental handovers, and administrative moderation/analytics.

1.3 Referenced Documents

- CampusConnect - Project Proposal (scope, technology stack, roles and responsibilities).
- CampusConnect - Project Plan (team timeline, module ownership, individual execution plans, dependencies, task board).
- CampusConnect - Requirements Specification / BRD (functional requirements FR-1.x to FR-7.x, non-functional requirements, user stories, acceptance criteria, low-fidelity wireframes).

1.4 Design Notation

All architecture, data, and behavioral diagrams in this document are authored in Mermaid syntax and rendered to images for inclusion here. Diagram source files (.mmd) and the machine-readable API contract (openapi.yaml) are maintained in the project Git repository alongside this document.

2. System Architecture

CampusConnect follows a service-oriented architecture built on the MERN stack, with a dedicated Python/FastAPI service for AI and matching workloads, Redis as an event bus decoupling producers (matching, rentals, moderation) from notification delivery, and Socket.io for real-time chat and live notifications. This mirrors the technology stack and module ownership defined in the Project Proposal and Project Plan.

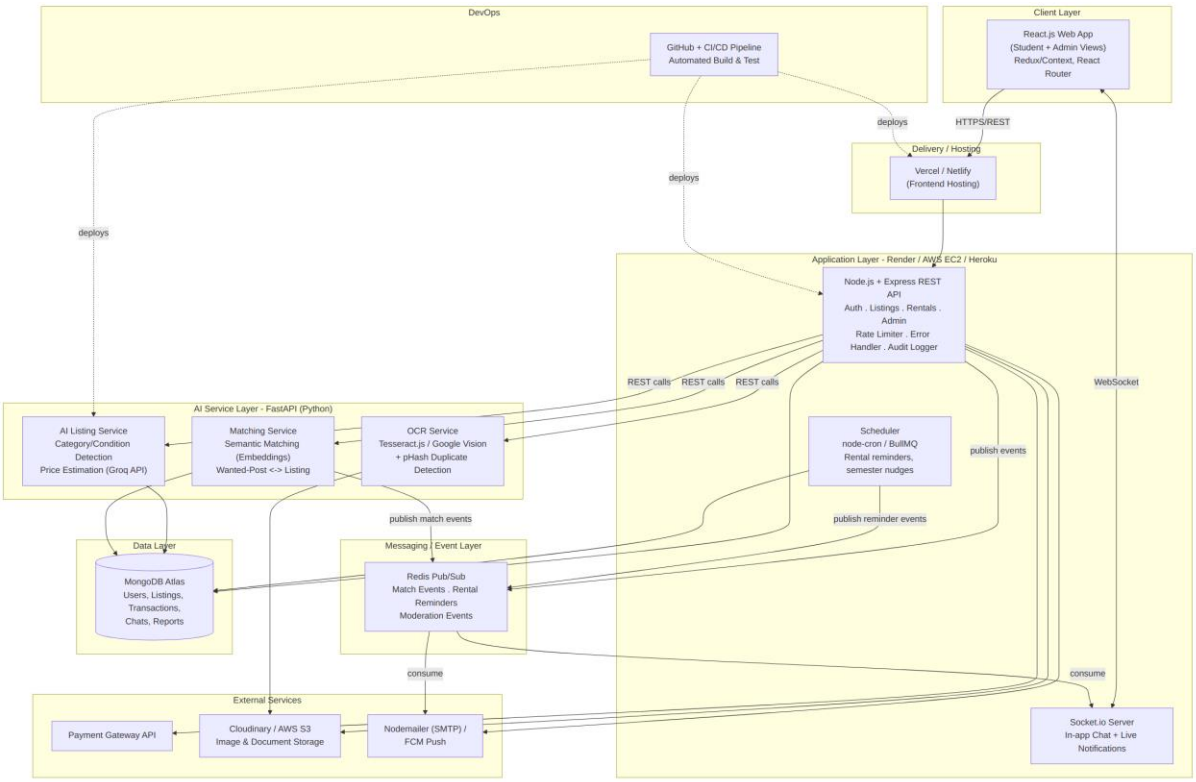


Figure 1: CampusConnect high-level system architecture.

2.1 Layer Descriptions

Layer	Responsibility
Client Layer	React.js SPA (Redux/Context, React Router) serving both Student and Administrator role-based views. Communicates over HTTPS/REST and a persistent WebSocket connection.
Delivery / Hosting	Frontend deployed on Vercel/Netlify; backend and AI services deployed on Render/AWS EC2/Heroku; MongoDB Atlas as managed database.
Application Layer	Node.js + Express REST API implementing Auth, Marketplace, Rental, and Admin business logic, wrapped by cross-cutting middleware: JWT auth guard, RBAC, rate limiter, global error handler, and audit logger (FR-7.1 to FR-7.3).
AI Service Layer	Python FastAPI microservice exposing versioned endpoints for AI-assisted listing suggestions, price estimation (Groq API), semantic matching (embeddings), and OCR-based duplicate detection, with mocked-response support for independent development (FR-3.5).
Messaging / Event Layer	Redis Pub/Sub decouples event producers (matching engine, rental scheduler, admin moderation actions) from the notification dispatcher and Socket.io delivery layer (NFR-2.2).
Data Layer	MongoDB Atlas (Mongoose ODM) stores users, listings, transactions, rentals, chat history, and reports.
External Services	Payment Gateway API, Cloudinary/AWS S3 for images and documents, Nodemailer/FCM for offline notification delivery.
DevOps	Git/GitHub with a CI/CD pipeline running automated build and test checks before deployment (FR-7.6).

3. Database Schema (ER Diagram)

CampusConnect uses MongoDB (document-oriented) with Mongoose schemas. The entity-relationship diagram below models the primary collections as relational entities for clarity of ownership and cardinality; in MongoDB, several 1:1 and small 1:many relations (e.g., Receipt within Transaction, SustainabilityStat within User) may be embedded rather than referenced, based on read/write access patterns.

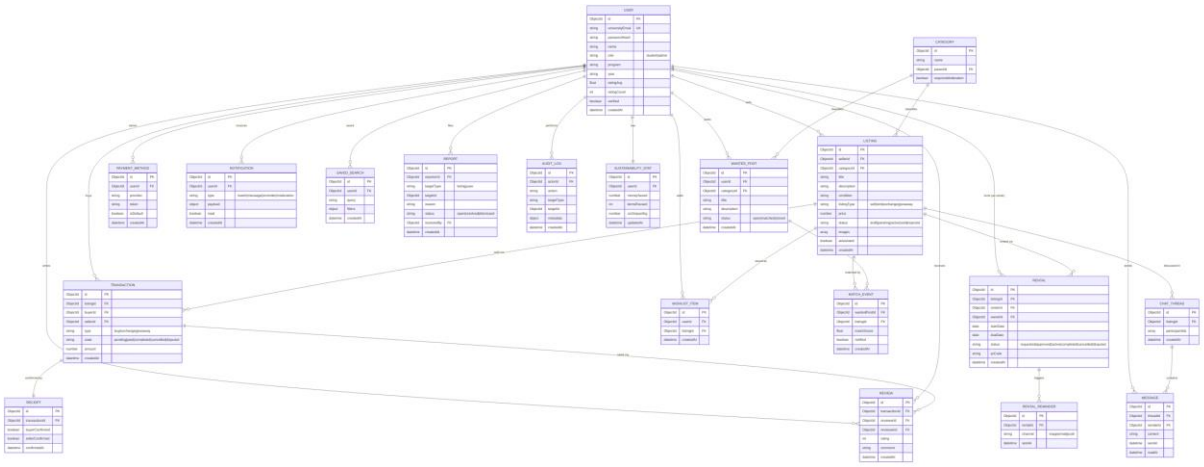


Figure 2: CampusConnect entity-relationship diagram.

3.1 Key Design Decisions

- User.universityEmail is unique-indexed and is the sole basis for registration eligibility (FR-1.1, NFR-3.4).
- Listing.status supports the moderation workflow (draft -> pending -> active -> sold/rejected) required by FR-6.1 and AC-7.
- MatchEvent is a derived/append-only collection populated by the Matching Service; it is never written to directly by clients (FR-3.3).
- Transaction and Rental are modeled as separate collections because a Rental has due-date and QR-handover semantics that a simple Transaction does not (FR-5.x vs FR-2.7).
- AuditLog and Report are append-only collections indexed by targetType + targetId for fast moderation lookups (FR-7.1, FR-6.4).
- ChatThread.participantIds and Listing references are indexed to support fast thread lookup per listing/user pair (FR-4.4).

4. Component, Class & Behavioral Design

4.1 Component Diagram

The component diagram below maps backend services to the module ownership defined in the Project Plan, showing how the platform core (owned by Dev Barot) fronts every module with shared auth, rate-limiting, error-handling, and audit-logging concerns.

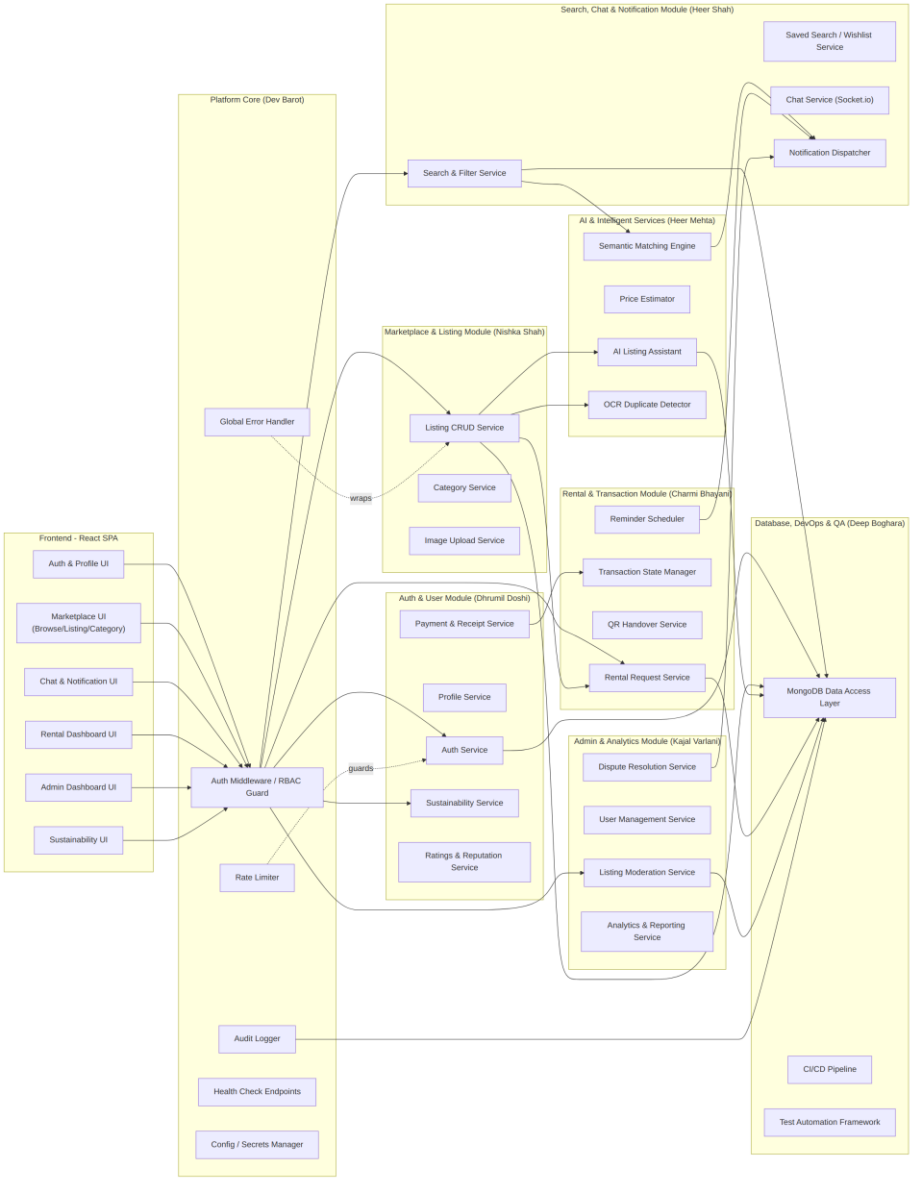


Figure 3: CampusConnect component diagram by module ownership.

4.2 Class Diagram

The class diagram captures the core domain model shared across modules: Users, Listings, Wanted Posts, Transactions, Rentals, Reviews, Chat, Notifications, and the AI/Admin service classes that operate on them.

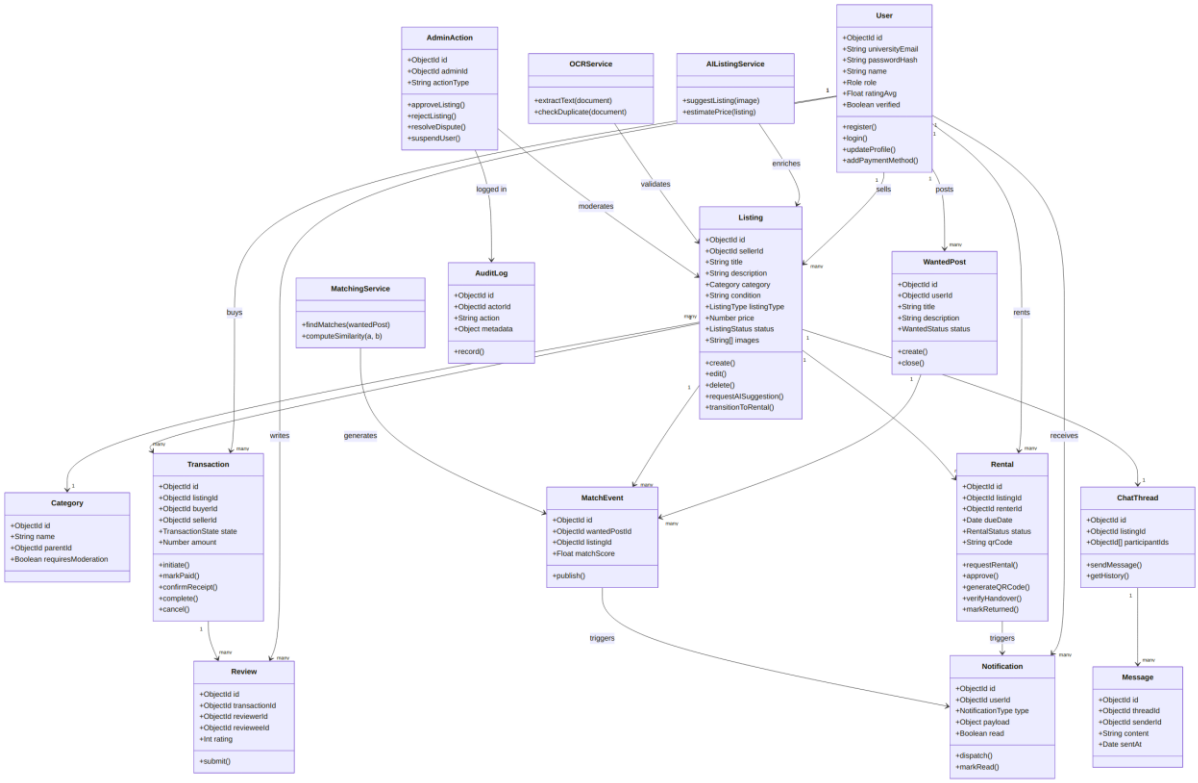


Figure 4: CampusConnect domain class diagram.

4.3 Sequence Diagram - AI-Assisted Listing Creation (US-03, AC-2)

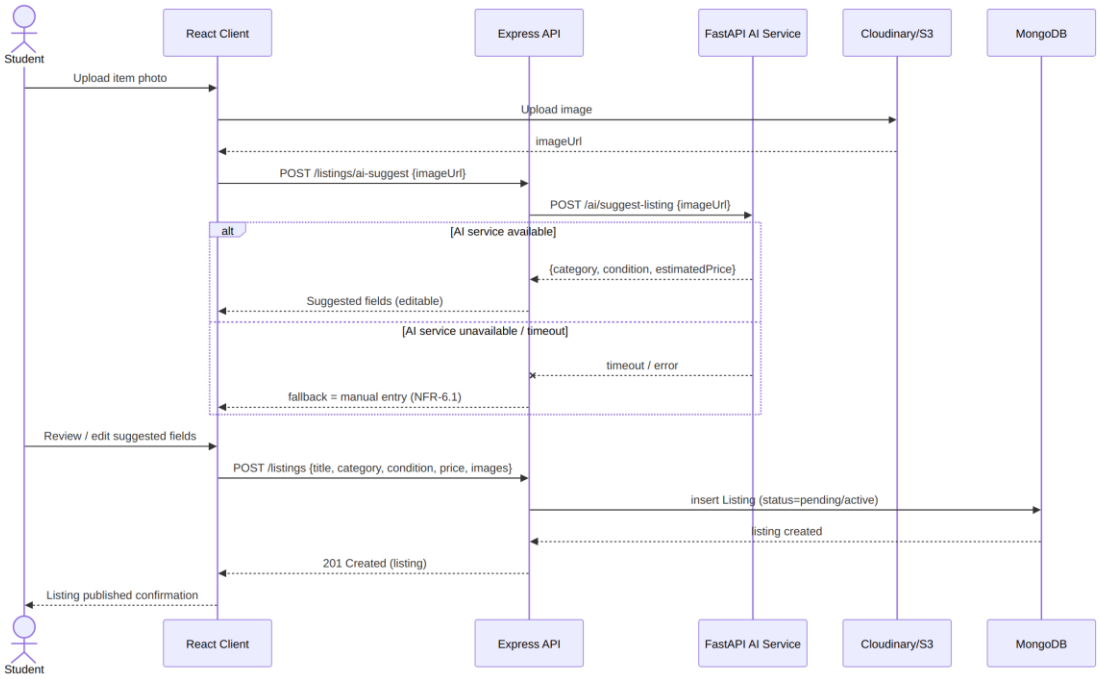


Figure 5: AI-assisted listing creation, including graceful fallback to manual entry (NFR-6.1).

4.4 Sequence Diagram - Wanted-Post Matching & Notification (US-05, AC-3)

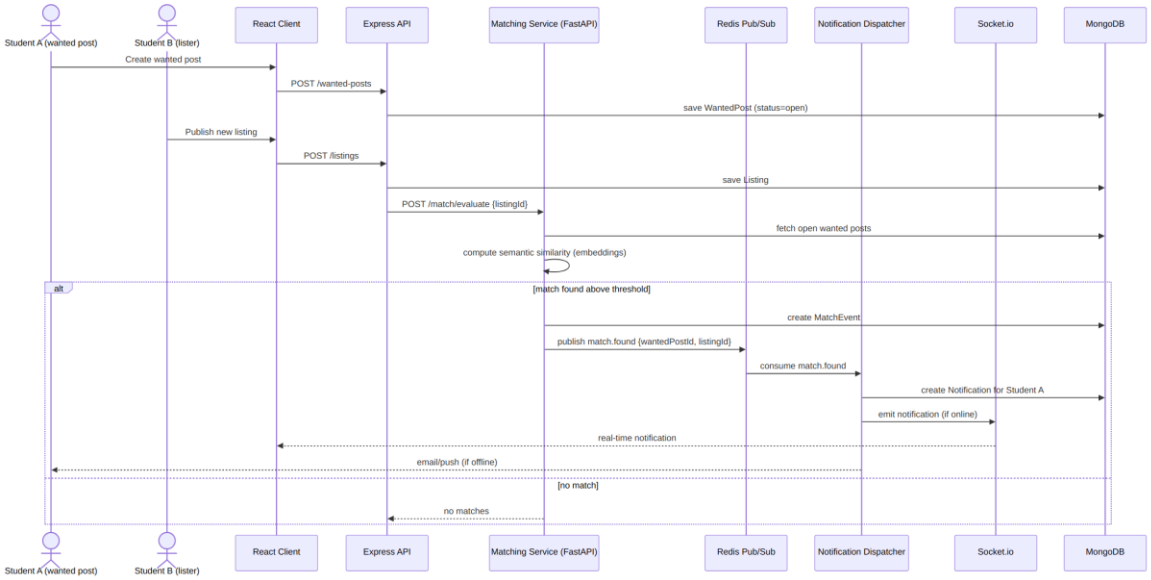


Figure 6: Semantic matching pipeline publishing match events through Redis to the notification dispatcher.

4.5 Sequence Diagram - QR-Verified Rental Handover (US-08, US-09, AC-5, AC-6)

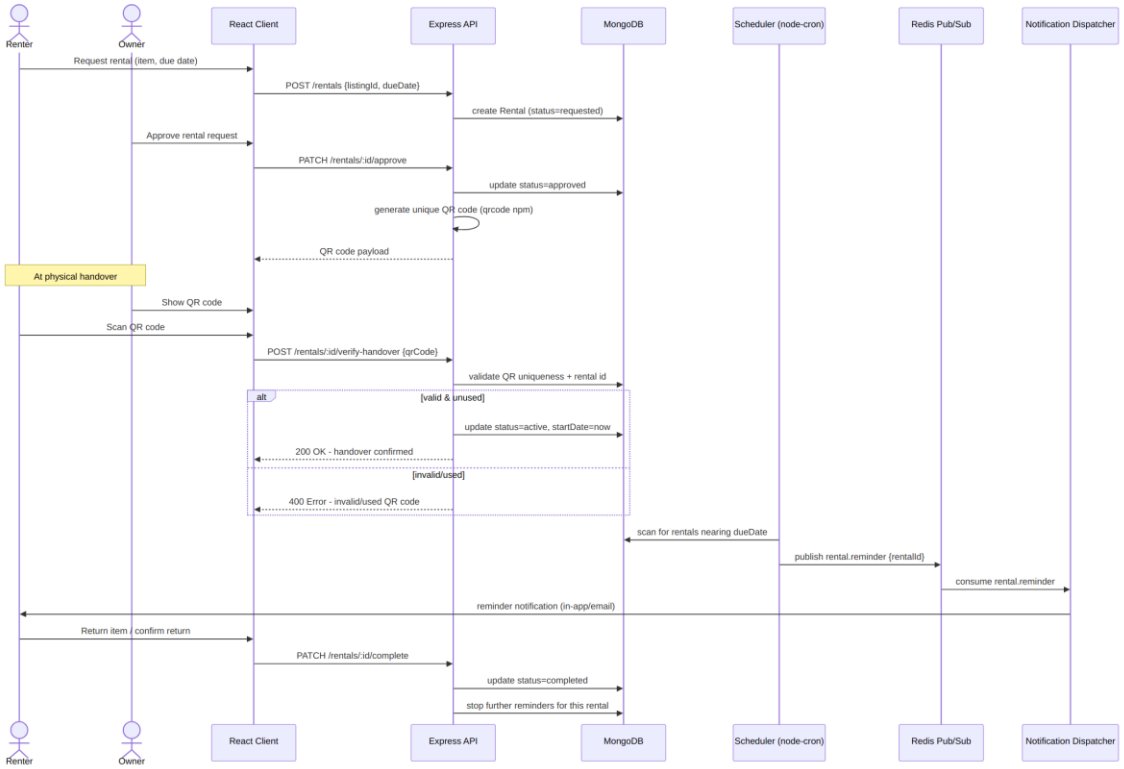


Figure 7: Rental request, approval, QR handover verification, and scheduled due-date reminders.

4.6 State Diagram - Transaction & Rental Lifecycle (FR-5.2)

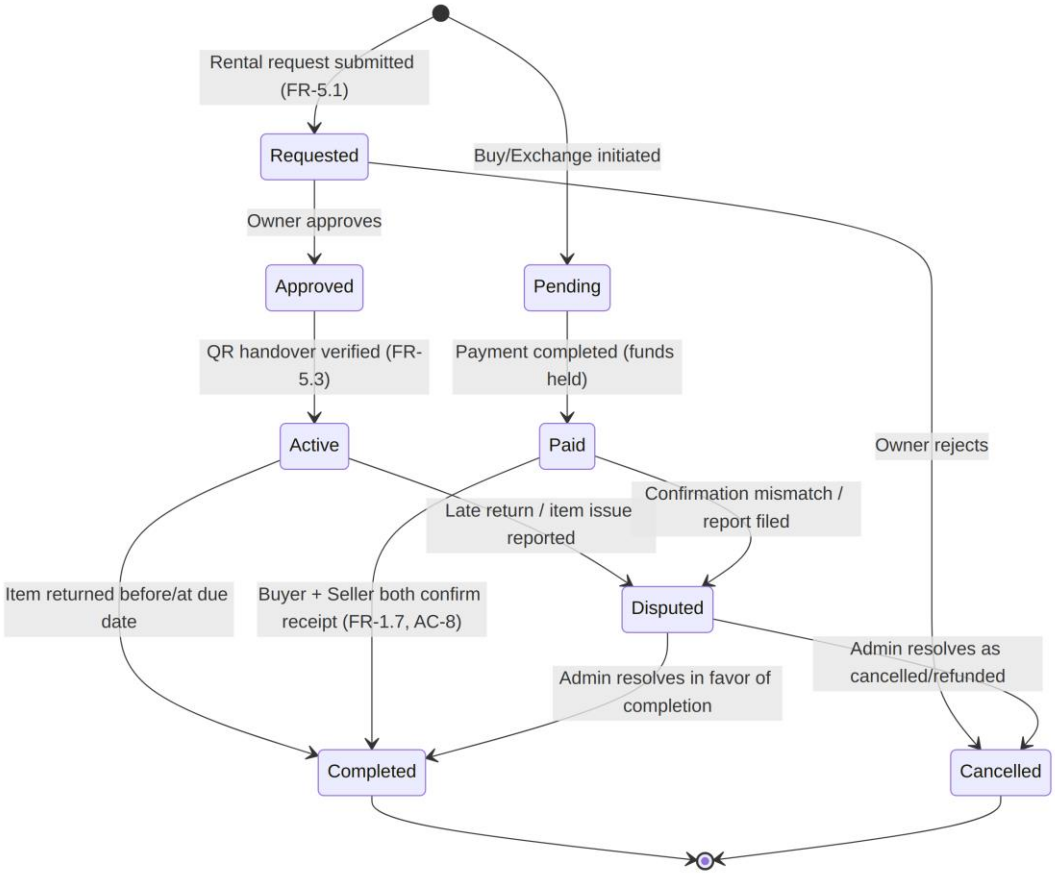


Figure 8: Combined state machine for buy/exchange transactions and rentals.

5. REST API Contracts

The full REST API is specified using OpenAPI 3.0 (openapi.yaml, included in the Git repository and Appendix A) so it can be rendered directly in Swagger UI / Redoc for interactive exploration and contract-first development across module teams (NFR-7.1). The table below summarizes the primary endpoints grouped by module.

5.1 Authentication & Users

Method & Path	Description	FR
POST /auth/register	Register with a verified university email	FR-1.1
POST /auth/login	Authenticate and issue JWT access/refresh tokens	FR-1.2
GET/PATCH /users/me	View / update profile	FR-1.4
GET/POST /users/me/payment-methods	Manage stored payment methods	FR-1.6
GET /users/{id}/reviews	Ratings & reputation for a user	FR-1.5
GET /sustainability/me	Personal sustainability dashboard data	FR-1.8

5.2 Marketplace & Listings

Method & Path	Description	FR
GET /listings	Browse/search with category, price, condition, type filters	FR-2.4
POST /listings	Create a new listing	FR-2.1
GET/PATCH/DELETE /listings/{id}	View, edit, or delete a listing	FR-2.1
POST /listings/{id}/images	Upload listing images to cloud storage	FR-2.3
GET/POST /categories	Manage marketplace categories	FR-2.2

5.3 AI & Intelligent Services

Method & Path	Description	FR
POST /listings/ai-suggest	AI category/condition/price suggestion from a photo	FR-2.5, FR-3.1, FR-3.2
POST /listings/{id}/ocr-check	OCR duplicate detection for notes/PYQs	FR-3.4
GET/POST /wanted-posts	Create/list wanted posts feeding the matching engine	FR-3.3

5.4 Search, Chat & Notifications

Method & Path	Description	FR
GET /search	Keyword search across active listings	FR-4.1
GET/POST /search/saved	Saved searches with match alerts	FR-4.2
GET/POST /wishlist	Manage wishlist items	FR-4.3
GET/POST /chat/threads	List / start chat threads	FR-4.4
GET/POST /chat/threads/{id}/messages	Message history / send (mirrored via Socket.io)	FR-4.4
GET /notifications, PATCH /notifications/{id}/read	Real-time & historical notifications	FR-4.5, FR-4.6

5.5 Rentals & Transactions

Method & Path	Description	FR
GET/POST /rentals	Rental dashboard / submit rental request	FR-5.1, FR-5.5
PATCH /rentals/{id}/approve	Owner approves; QR code generated	FR-5.2, FR-5.3
POST /rentals/{id}/verify-handover	Verify QR at pickup/return	FR-5.3, AC-5
PATCH /rentals/{id}/complete	Mark rental completed; stop reminders	FR-5.4
POST /transactions	Initiate buy/exchange/giveaway transaction	FR-2.7
POST /transactions/{id}/pay	Process payment (funds held)	FR-1.6
POST /transactions/{id}/confirm-receipt	Buyer/seller receipt confirmation	FR-1.7, AC-8
POST /reviews	Submit rating/review post-transaction	FR-1.5

5.6 Admin, Analytics & Platform

Method & Path	Description	FR
GET /admin/listings/pending, PATCH .../approve|reject	Listing moderation queue	FR-6.1, AC-7
GET /admin/users, PATCH /admin/users/{id}/suspend	User management	FR-6.2
GET /admin/disputes, PATCH .../resolve	Dispute resolution workflow	FR-6.4
GET /admin/analytics	Demand trends, active users, listing volume	FR-6.5
POST /reports	Report inappropriate listing/user	FR-6.4
GET /health	Per-service health/uptime endpoint	FR-7.4
GET /audit-logs	Query audit trail (admin only)	FR-7.1

All authenticated endpoints require a Bearer JWT (HTTP Authorization header) issued by /auth/login, validated by the shared Auth Middleware described in Section 4.1. Full request/response schemas, status codes, and the AI-service fallback contract (503 -> manual entry) are defined in openapi.yaml (Appendix A).

6. UI/UX Mockups (Final)

The following higher-fidelity mockups refine the low-fidelity wireframes from the Requirements Specification (Section 6) into the near-final visual design: a navy/teal brand palette, card-based layouts, and consistent iconography across student and admin views.

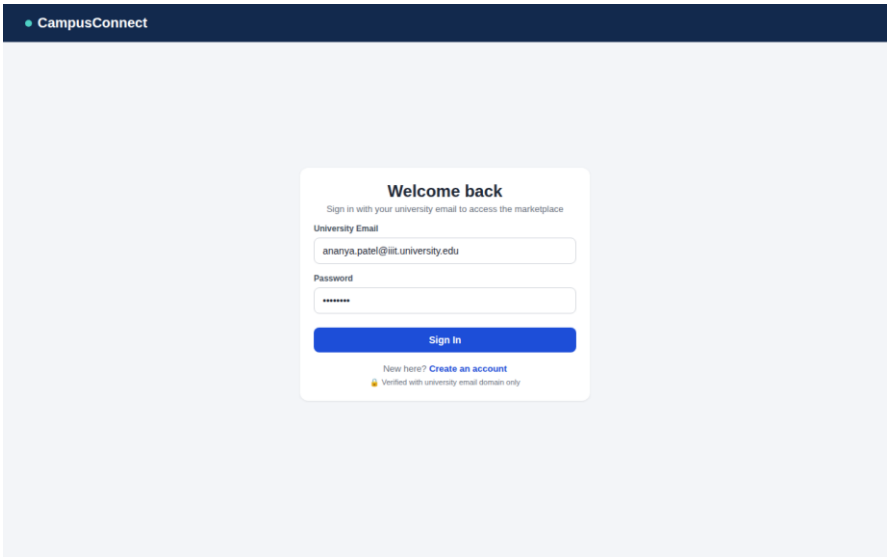


Figure 9: Login / Registration - university-email-gated entry point (FR-1.1, FR-1.2).

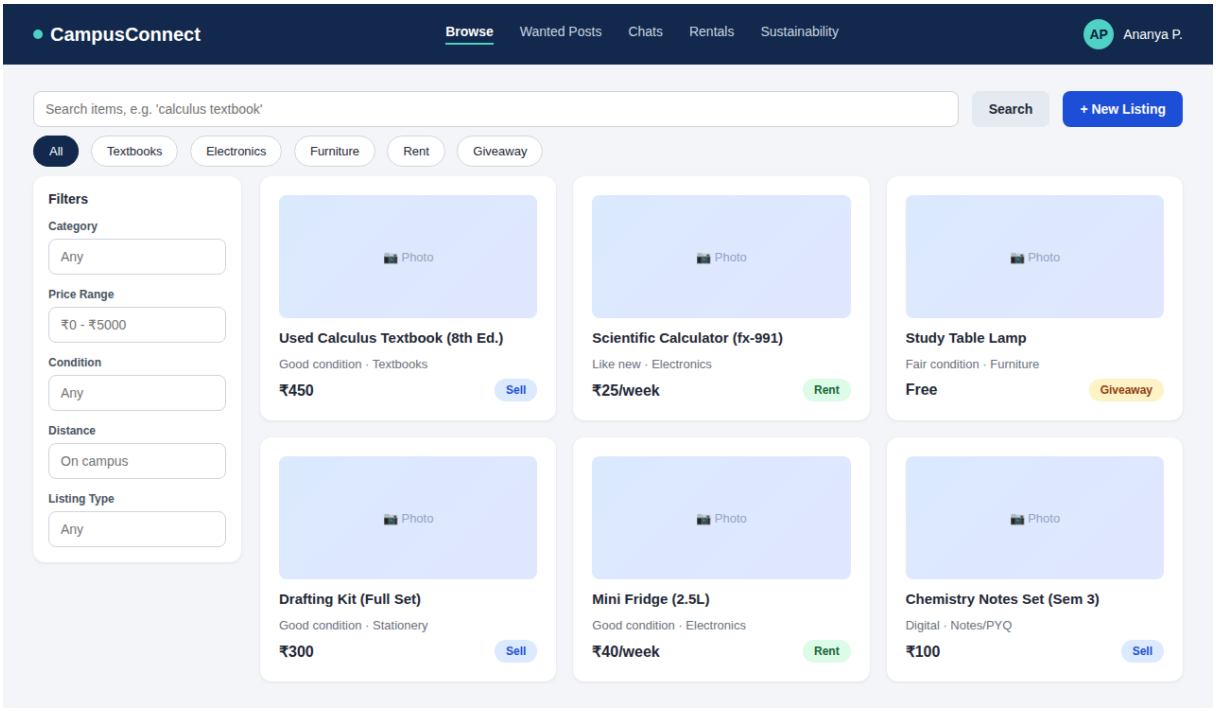


Figure 10: Browse Marketplace with category chips and advanced filters (FR-2.4).

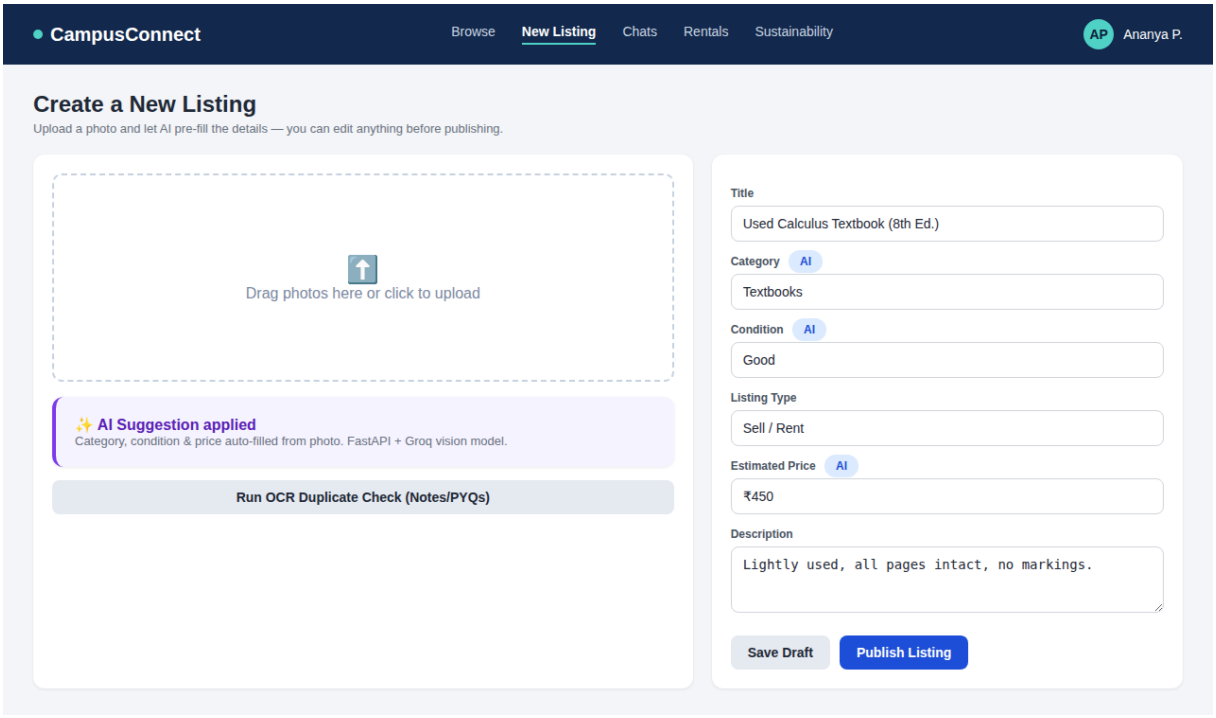


Figure 11: AI-Assisted Create Listing with editable AI-suggested fields and OCR check (FR-2.5, FR-3.1, FR-3.2, FR-3.4).

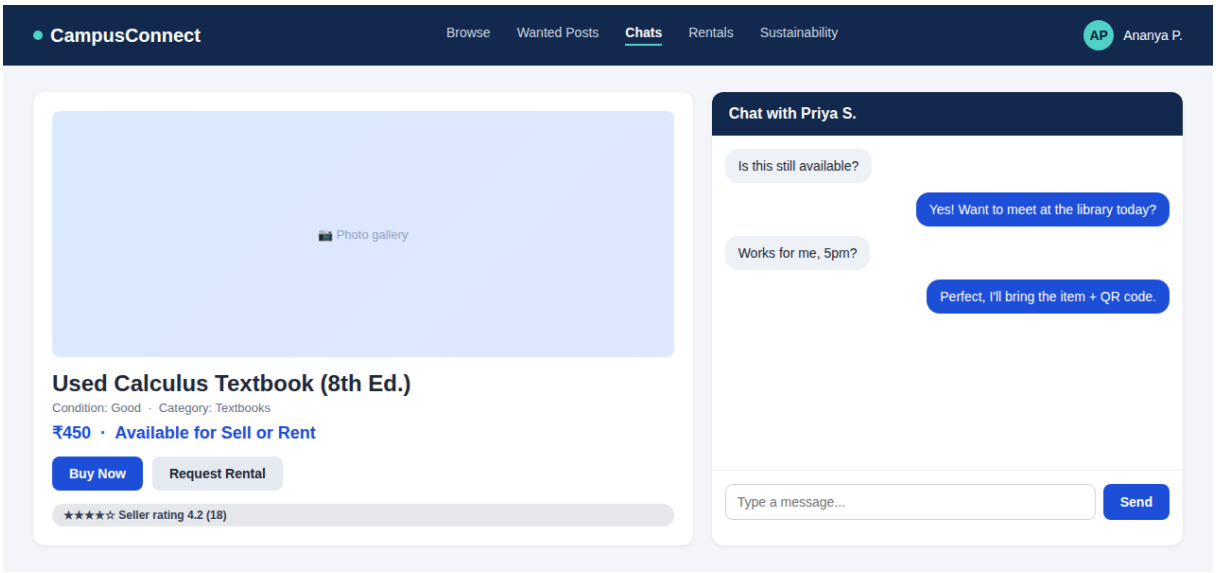


Figure 12: Listing Detail with integrated real-time chat panel (FR-4.4).

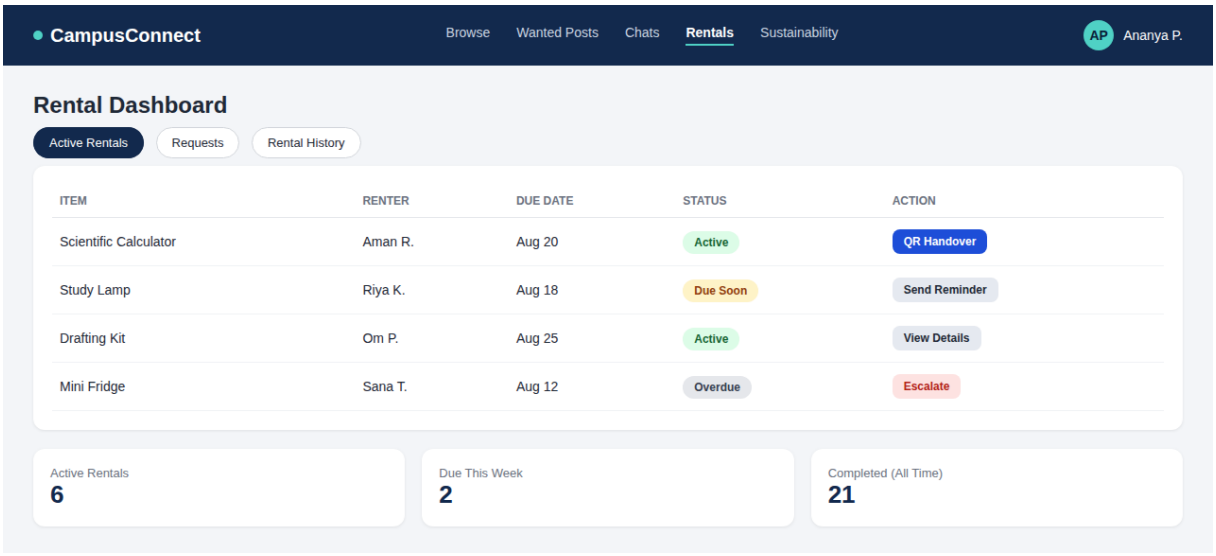


Figure 13: Rental Dashboard with due-date status and QR handover action (FR-5.3, FR-5.4, FR-5.5).

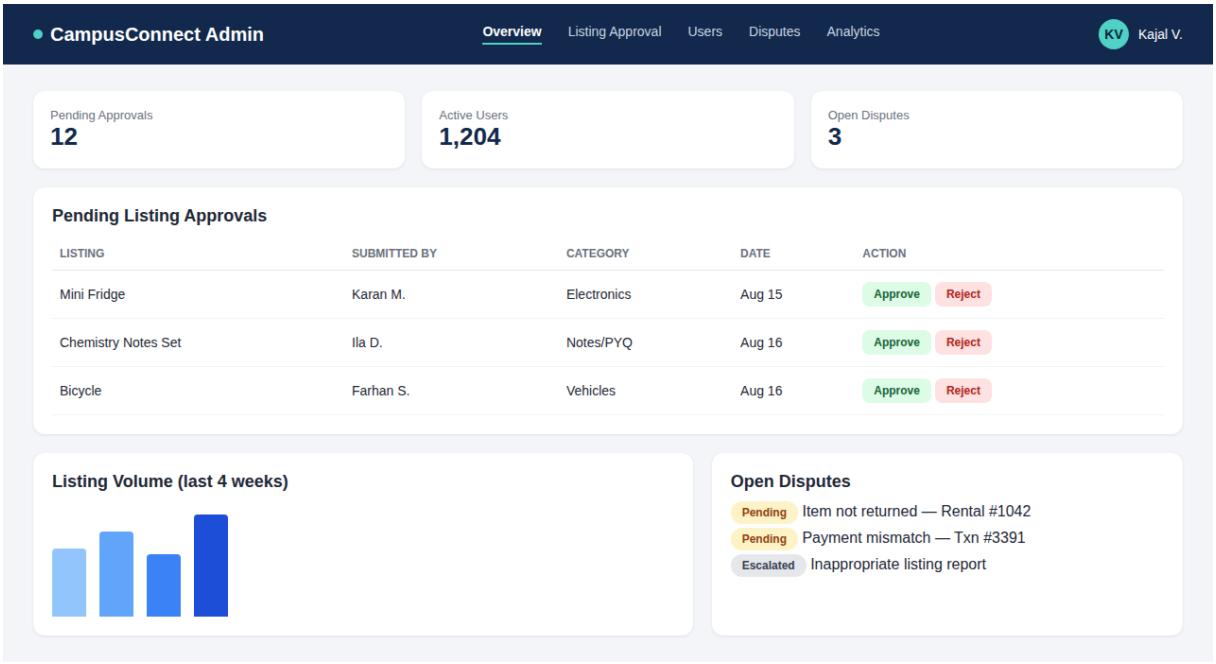


Figure 14: Admin Dashboard - moderation queue, dispute list, and listing-volume analytics (FR-6.1, FR-6.4, FR-6.5).

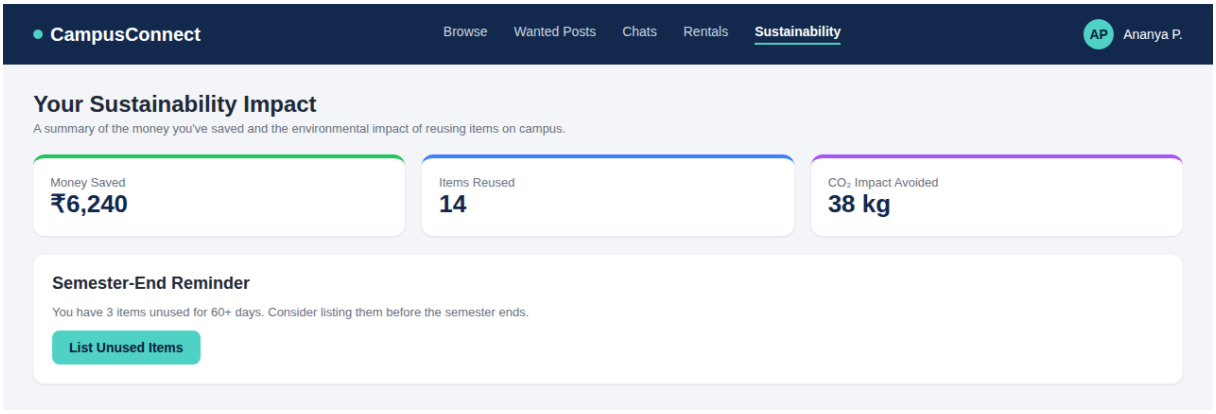


Figure 15: Personal Sustainability Dashboard (FR-1.8).

7. Cross-Cutting & Non-Functional Design

Quality Attribute	Design Approach
Security	JWT + bcrypt auth, RBAC guard on every route, HTTPS/TLS everywhere, input validation/sanitization, secrets via environment config (NFR-3.1 to NFR-3.5).
Performance	Indexed MongoDB queries for browse/search (<1s for 10k listings), Socket.io for <500ms chat delivery (NFR-1.1 to NFR-1.3).
Scalability	Stateless Express + FastAPI instances behind load balancers; Redis Pub/Sub decouples producers/consumers so AI, matching, and notification services scale independently (NFR-2.1, NFR-2.2).
Reliability	AI-service outage falls back to manual listing entry without blocking the user; payment/transaction state remains consistent even if notification delivery fails (NFR-6.1, NFR-6.2).
Availability	Health-check endpoints on every major service enable automated monitoring and alerting; 99% uptime target during the academic semester (NFR-4.1, NFR-4.2).
Maintainability	Shared OpenAPI contract across module teams; automated unit tests wired into the CI pipeline for every module (NFR-7.1, NFR-7.2).
Auditability	Centralized audit-logging middleware records key user/admin actions for accountability and dispute investigation (FR-7.1, NFR-9.1).

Appendix A: OpenAPI Specification

The complete machine-readable REST API contract is maintained as openapi.yaml in the project Git repository and submitted alongside this document. It can be opened directly in Swagger Editor (https://editor.swagger.io) or rendered via Redoc for interactive browsing. Section 5 of this document summarizes its contents by module.

Appendix B: Diagram Source Files

All diagrams in this document are authored in Mermaid (.mmd) syntax and version-controlled in the Git repository under /design/diagrams, alongside the rendered PNG images embedded here:

- 01_architecture.mmd - System architecture
- 02_er_diagram.mmd - Database ER diagram
- 03_component_diagram.mmd - Component diagram
- 04_class_diagram.mmd - Domain class diagram
- 05_seq_ai_listing.mmd - AI-assisted listing sequence
- 06_seq_matching.mmd - Wanted-post matching sequence
- 07_seq_qr_handover.mmd - QR rental handover sequence
- 08_state_transaction.mmd - Transaction/rental state diagram